Towards Sustainable Computing: Exploring Energy Consumption Efficiency of Alternative Configurations and Workloads in an Open Source Messaging System

Maria Voreakou*, George Kousiouris, Mara Nikolaidou
Harokopio University of Athens, Athens, Greece
{voreakou, gkousiou, mara}@hua.gr

***Abstract*—Energy consumption in current large scale computing infrastructures is becoming a critical issue, especially with the growing demand for centralized systems such as cloud environments. With the advancement of microservice architectures and the Internet of Things, messaging systems have become an integral and mainstream part of modern computing infrastructures, carrying out significant workload in a majority of applications. In this paper, we describe an experimental process to explore energy-based benchmarking for RabbitMQ, one of the main open source messaging frameworks. The involved system is described, as well as required components, and setup scenarios, involving different workloads and configurations among the tests as well as messaging system use cases. Alternative architectures are investigated and compared from an energy consumption point of view, for different message rates and consumer numbers. Differences in architectural selection have been quantified and can lead to up to 31% reduction in power consumption. The resulting dataset is made publicly available and can thus prove helpful for architectures' comparison, energy-based cost modeling, and beyond.**

***Index Terms*—Sustainable Computing, Messaging Systems, RabbitMQ, Energy Consumption, Testbed, Sustainable Architectures, Open Energy Dataset**

I. Introduction

Sustainability and Green computing [29] are important topics of the current era, with multiple and complex applications across large data centers, consuming more and more energy as an integral part of our digital life. Data centers' energy consumption has started to become an issue in recent years [20] [30]. Therefore, careful consideration needs to be applied on the design, configuration, and operation of systems and software under various loads, in order to reduce energy consumption, by closely monitoring and analyzing different strategies, optimizations, and understanding the behavior of such systems.

Messaging systems have emerged as one of the main backbones of modern application and system architectures. Messaging systems act as a main backbone interconnecting different systems, in a System of Systems approach, in order to enforce greater separation, enhance scalability and fault tolerance, as well as enable abstraction and functional separation between systems. They are extensively used either as data distribution systems (e.g. in IoT platforms [34]), generic load balancing mechanisms (e.g. in the case of the queue-based load leveling pattern [31]), as core cloud services (e.g. Amazon SNS/SQS) or as the main mechanisms in social networks [32]. Therefore, they are highly relevant in a distributed, large-scale, and cloud native context, carrying out significant workload in day-to-day activities. They also ensure message delivery in faulty environments with out of the box functionalities for message tracking, delivery guarantees and cross-system notifications.

The scope of this study is to extract the energy footprint of one of the most commonly used open source messaging systems, RabbitMQ, under varying incoming and outgoing loads, represented through the parameters of input messages per second, number of routing keys as well as number of consumer queues. Furthermore, we take into consideration different strategies in its configuration, such as the selection of the main type of exchange for the same usage scenario.

RabbitMQ consists of 3 different exchanges: *Direct*, *Topic*, and *Fanout* Exchange, which define the way the messages are distributed to the consumers through the queues. *Direct* creates a simple Exchange with a strict filtering pattern on the message, so the consumers subscribed to this exchange receive all the messages sent to this queue that exactly match the desired routing key. Similarly, *Topic* creates an exchange with a partial or full match filtering pattern, and the consumers subscribed to the specific topic can consume the messages from the bound queues, if the partial or full match is successful. *Fanout* on the other hand is an exchange which broadcasts the message to all the queues that are bound to this exchange, so all the consumers listening to any queue of this exchange will consume all the messages.

The goals of this experiment are as follows:

- Create a testbed architecture that includes the main components (load generator, RabbitMQ setup, energy monitoring, etc.) and define scenarios to explore, by setting parameters such as message rate, Queues, Routing keys, etc.
- Map the different experimentation parameters on the energy needed for the operation of the system. The analysis

Part of the research leading to the results presented in this paper has received funding from the European Union's funded Project HUMAINE under grant agreement No. 101120218.

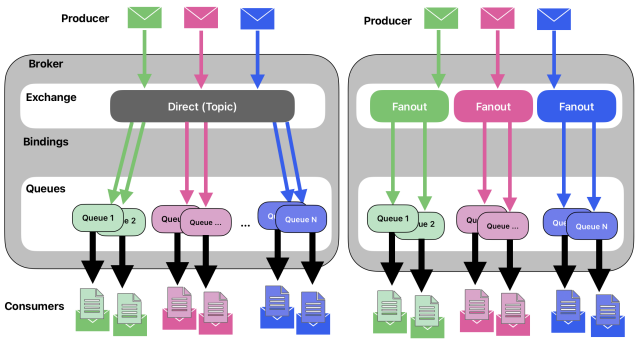

Fig. 1: RabbitMQ Direct vs Fanout Exchange Experiment Strategy

of such data may result in understanding how these parameters affect the total energy consumed. Having that information may result in future specific energy-based billing models for messaging systems offered as a service, i.e. in cloud-based environments.

- Quantify the effect of choosing between two different strategies in order to distribute messages based on routing keys, as shown in Figure 1. While the typical strategy within RabbitMQ is to use one topic-based exchange with full key matching or a direct exchange, an alternative design may be considered in which each key is represented by a separate fanout exchange. In this case, all interested consumers are bound to each of the desired exchanges. With this, we achieve eliminating routing key matching, and thus we could potentially see different behavior in terms of energy consumption. As a trade-off, we need to have a more complex registration process and potentially larger memory needs due to the multiple exchanges' design.

Based on the overall results and the relevant conclusions, a systems engineer should be able to define the best possible setups so that the application created is both more sustainable and better performing.

The paper is structured as follows. In Section 2, the relevant works are mentioned, in terms of relevant tools and analysis. In Section 3, we present the Project Testbed with the selected tools, including hardware and software setup, the test application and the experiment scenarios we are going to run. In Section 4, we present an initial analysis of the results including the main observed patterns and trends. Finally, in Section 5, we conclude the analysis and define future steps.

II. Related Work

Authors of [24] built a decentralized MQTT for a real-time remote healthcare monitoring system using local edge devices, and measured the energy consumption using the *turbostat* tool, to determine the overall system energy footprint. No special information regarding the RabbitMQ energy consumption is mentioned. However, they concluded that RabbitMQ has proved resilient in managing data during network disruptions, ensuring uninterrupted patient monitoring.

Authors of [27] made an analysis in the current carbon estimation methodologies and proposed a usage and allocation based carbon model. Although this model aligns only with serverless computing, they concluded that it is valuable to design models based on carbon-aware pricing which is influenced directly by the power consumption. To enforce such approaches on messaging systems, the authors proposed that relevant benchmark data such as the ones presented in this work are needed.

Authors of [21] evaluated existing energy models (*PowerAPI* [22] and *Scaphandre* [17]) in servers using different performance settings such as hyperthreading [8] and turbo-boost [9] to measure the overall system usage while processing different applications.

Authors of paper [23] evaluated their Radio Access Networks (RANs) using *Scaphandre* [17] and *Kepler* [10] by measuring energy metrics, in order to build power-aware scheduling algorithms.

In [33], the author motivated by the ever-increasing energy consumption in data centers, measured the power consumption in applications written in C, by using *Scaphandre* [17] to compare different algorithms and their power consumption.

Authors of [25] presented an overall evaluation of tools used to measure energy consumption including Power Profiling Software, Energy Measurement Software, as well as Energy Calculators. For real-time process-level measurements, Power Profiling software such as *Scaphandre* [17] is able to estimate more processes running in parallel and it has a less complicated and more intuitive setup process for working in a virtualized environment [25]. The authors concluded that software-based power meters based on Intel RAPL [26] and NVIDIA NVML [12] can be used to estimate consumption at a fine granularity with low overhead.

Open Source tools such as *OpenMessaging* [14] provide an overall benchmark solution including the most famous MQTT brokers, however they do not include relevant metrics available for power consumption.

III. Energy Consumption Exploration Testbed

In order to perform the envisioned experiments, a testbed is constructed, including the target application that contains a messaging system, target hardware, load generation processes, and means for collecting the relevant energy and hardware metrics.

A. Test Application

The test application is a Cloud/Fog application [19] [18] that includes an edge computing component to obtain sensor readings, and a cloud backend component for data ingestion based on RabbitMQ [16]. The application consists of 3 main entities as shown in Figure 2, the *Backend* which is the RabbitMQ backend, the *Producers* (sensors) which are responsible to send the data to RabbitMQ, and the *Consumers*. The backend also includes a MongoDB, which stores data as a part of a reliability and availability of the sensors (e.g. heartbeat of a sensor) scheme. The goal in this study is to analyze the backend part, and specifically the RabbitMQ component.

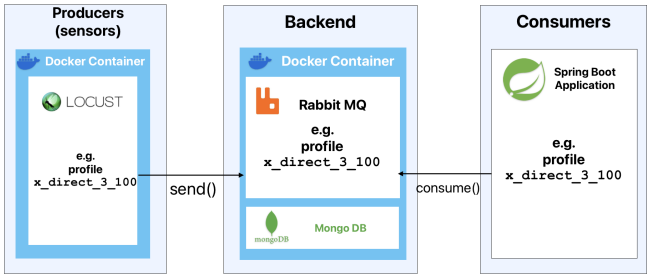

Fig. 2: Testbed Overview

B. Hardware Setup

The involved hardware consisted of two nodes, one for the execution of the traffic load (production and consumption clients) and one for the main backend. The separation is necessary in order to keep the acquired backend measurements as pristine as possible, since load generation and consumption is by definition a very intensive process that can lead to interference. The Hardware specifications of the node responsible to run the backend are: CPU Intel Core i7-4790K (up to 4.4GHz), 32 GB of DDR3 1600MHz RAM, 250GB Hard disk storage, running in Ubuntu OS 24.04 LTS.

C. Power Measurement Software

To be able to measure the energy consumption on a computing system, a way is needed to get the metrics on the bare metal of the system. Libraries related with these kind of metrics are few, and some examples are *Kepler* [10], *dhsb* library [2], *energy-tracker* library [5], *Open Hardware Monitor* [13], and *powerstat* [3]. Although the aforementioned libraries give an overview about energy consumption, they do not provide more fine grained measurements, such as power consumption per process ID. *PowerAPI* [22] and *Scaphandre* [17] are Power-Profiling software, obtaining information from CPU & RAM directly using RAPL. RAPL stands for ”Running Average Power Limit”. It is a feature on Intel & AMD x86 CPU’s manufactured after 2012 which provides power measurements at a very fine-grained level [17] [26]. Both *PowerAPI* and *Scaphandre* are commonly used for real-time measurements.

Both *PowerAPI* and *Scaphandre* are giving quite similar results compared to other existing tools [21]. However, *PowerAPI*’s features like Smart-watts, which is based on performance event-based regression modeling, may affect in some cases the reliability of the power consumption measurements. Furthermore, *Scaphandre*’s ability to run more parallel processes with less overhead [25] made us select it as our tool of preference. Based on the model *Scaphandre* uses, it calculates micro-watts per process ID. It also gives the option to run it on bare metal, or to use it with Kubernetes, and still measure metrics on bare metal. It has also been used in research to measure power consumption in data centers to reduce negative impacts such as energy waste [33].

To analyze the energy and performance profile of RabbitMQ, a number of different metrics have been used, such as CPU, Memory, Disk, and traffic usage. The project’s goal is to measure the energy consumption in different scenarios, getting a measured result of what are the best practices in order to achieve a more sustainable system.

In order to use *Scaphandre* on our system, some configuration needs to take place. The steps taken are documented on the following GitHub repository [7], for validation and reproduction purposes. *Scaphandre* uses a *Prometheus* exporter [28], and also comes dockerized together with a *Grafana* dashboard [6], which helps to see in real-time graphs and results of the energy consumption of the application under investigation.

D. Measurement Scenarios

Measurement scenarios revolve around the main messaging system (RabbitMQ). It is reasonable to consider that energy usage may be affected by different factors such as the rate of incoming messages, the exchange type, the number of consumers, as well as how the messages get delivered from source to destination. RabbitMQ is based on the concept of routing keys. Each incoming message may be annotated by such a key that contains relevant metadata. Then a consumer can have their queue bound to an input (exchange), based on the needed metadata matching. Thus, based on the usage scenario, a given incoming message may be distributed to one or multiple consumers based on these bindings between queues and routing keys.

In our work, we have considered two major messaging use cases.

- *Account driven use case*: The scenarios consist of the same number of routing keys as the Queues (i.e. a 1:1 ratio between incoming and outgoing messages). Scenarios like this can be applicable to account based messaging where each account consumes only the messages that are specifically targeted to them, with the routing key being the user ID for example. Such systems may include e-government, e-commerce systems for ordering, banking systems etc.
- *Data Distribution driven use case*: The scenarios consist of a limited number of routing keys that interest multiple consumers (queues) per key. Thus an incoming message may be delivered to *M* interested consumers, having a 1:M ratio between incoming and outgoing messages. This use case might be applicable to scenarios such as sensor/event data delivery in IoT platforms, Priority Messaging filtering (each key refers to different priority or type of information level), group messaging filtering (when many subscribers consume the same information), and so on.

For the examined use cases, and in order to implement the different design mentioned in the Introduction, we can either use a direct exchange or a topic one with a full key matching for comparing with the fanout strategy. In our case, we selected the latter so that the system is flexible to be extended in the future with partial matching use cases.

The naming structure of the project scenarios is defined based on the parameters on which we want to

experiment. First, the Incoming **Message Rate** per **Second**, the **Exchange** Type this scenario relies on based on the strategy (direct/fanout) identified in Figure 1, the defined Number of **Routing Keys** (or the **Number of Exchanges** in case of a fanout scenario), as well as the Number of **Queues**. The final naming structure is: **MessageRate_Second_Exchange_RoutingKeys_Queues**

All the project scenarios created for this research are mentioned in Table I. In addition to these parameters, we have also divided the scenarios under the two different use cases (*Account* versus *Data Distribution*) to match to realistic usages. We take as granted that the Exchanges are Durable [1], a RabbitMQ option that guarantees delivery and poses significant burden on the system such as increased disk I/O.

On our testbed, we have configured both Queues and Exchanges as Durable, which leads to messages to also be stored on RabbitMQ's side, in case a consumer is unable to consume messages on time. Given this, consumers may also impact the behavior of RabbitMQ, depending on their availability and how fast they consume the messages from their queues. Messages that are in the "Ready" state (i.e. messages delivered to a consumer queue but not yet consumed by the client) are stored in the disk and memory of RabbitMQ. So a more "lazy" consumer will pose more stress on the system.

In order to include also this additional energy impact, we assumed in our implementation that consumers are not always available to receive all the messages immediately.

E. Message Generation and Consumption

In order to make the scenarios easy to run, as well as to get reliable results, various metrics across the system must be monitored. During and at the end of each experiment, we have been checking metrics related with the messages produced, message rates, RabbitMQ memory and disk usage, and metrics provided by Scaphandre, such as CPU & Memory usage per PID, and more. Some of the metrics we find important for this paper, are indicated in Table I.

The automations and settings that were applied are:

RabbitMQ Configurations: The following configurations were made per scenario inside the code using SpringBoot @Profiles [15]:

- dynamic configuration of queues, bindings, and exchanges through the use of application.yml variables
- generate consumers using @Listener to dynamically create them
- adjust the Task Executors so that the mocked consumers consume in a similar rate as in real life scenarios

Usage of *Locust* library: Locust is an open source load testing tool [11], used to make our load tests real-like. By using the pika library [4], and by defining 1 worker and 1 User per 1 Task, we had the number of users be the same as the number of messages that will be published in RabbitMQ, thus acting as a publishing client.

Command Line Access: In order to avoid UI access overheads, which can be significant on RabbitMQ for such message rates, all accesses were terminal-based. The RabbitMQ and Grafana UIs were accessed only at the beginning and the end of each experiment.

$$\text{Total Messages } n_{em} = n_{rate} \cdot t \cdot \left(\frac{n_{queues}}{n_{keys}} \right) \quad (1)$$

Eq. 1 Calculation of the total messages processed

$$\text{Consumed Messages} = n_{em} - n_{rm} \quad (2)$$

Eq. 2 Calculation of the messages consumed

System split in 2 nodes: The *Backend* entity is installed on a different node than the *Producers* and the *Consumers* of RabbitMQ messages, as shown in Figure 2, in order to not interfere with the energy results.

IV. Results

In this section we analyze the effect of the various parameters (load rate, number of keys, number of queues) on the energy consumption of each type of exchange setup. In order to compare the results of all the scenarios, we first need to define the values we examine, and how these will get us the conclusion for the best setup per use case.

To be able to measure the energy consumption at a message level, but also validate the experimental process, we calculate the total messages processed per experiment, as shown in Equation 1, where n_{rate} is the defined input message rate (messages/second), t is the total time of the experiment, n_{queues} is the number of queues and $n_{routing\ keys}$ is the number of routing keys.

We also define the number of messages consumed as shown in Equation 2, where n_{em} is the total messages of the experiment and n_{rm} is the number of messages on the state *Ready* in RabbitMQ at the end of the experiment. The latter indicate the finalization of their routing inside RabbitMQ and their delivery to the respective queues.

A. Analyzing the results by Use Case

All the experiments ran in discrete and separate 30-minutes slots. A dataset was created and is accessible through a GitHub Repository [7], for validation and reproduction purposes. In Table I, some of the exported metrics are mentioned, in order to better describe the analysis in the rest of this Section.

1) Account Driven Use Case: In Figure 3, the results of power consumption of all the scenarios of this use case is shown. All the scenarios in this use case consist of exactly the same number of routing keys as the number of Queues, which are either 100, 500 or 1000. The Figure is also divided into 2 different message rates, one for 100 messages/second and one for 300 messages/second. In general, the results show that the *Fanout* strategy had on average power consumption from 4,58% up to 20,28% lower than *Direct*, while the *Direct* always had a slightly higher power consumption. Furthermore, we see a correlation between Energy Consumption and average Memory usage, as shown in Table I. All scenarios related with the *Fanout* strategy have a lower memory usage in average,

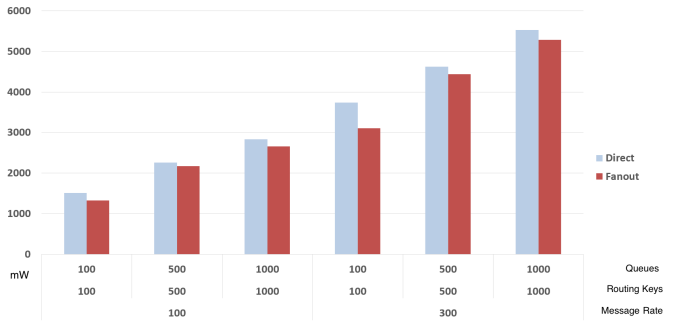

Fig. 3: Account Driven Uses Cases comparison of average Power Consumption in mW

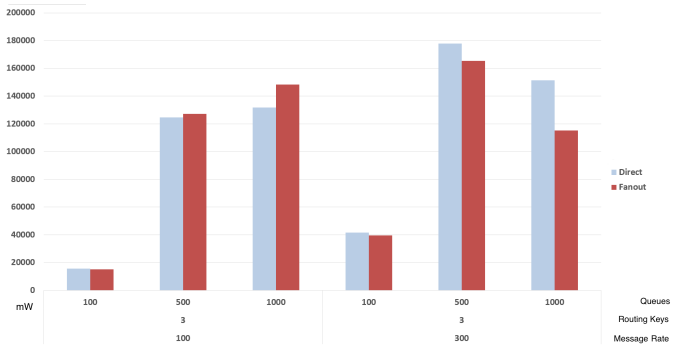

Fig. 4: Data Distribution Driven Uses Cases comparison of average Power Consumption in mW

while all the *Direct* scenarios have slightly increased memory usage. We can also see in Table I, that no Disk access (Read and Write access) is performed during any of the scenarios of this use case. This might be due to the fact that this use case's Outgoing/Incoming factor is 1, which means that each Queue is bound with just one routing key, and thus there is no additional message reproduction needed from RabbitMQ, as the message is sent to exactly one exchange.

2) Data Distribution Driven Use Case: In Figure 4, the power consumption of all the scenarios of this use case is shown. All the scenarios in this use case consist of exactly 3 routing keys, while the number of Queues is either 100, 500 or 1000. The Figure is also divided into 2 different message rates, one for 100 messages/second and one for 300 messages/second. In general, the results show that while for the scenarios with 100 messages/second, *Fanout* has a slightly increased average power consumption (from 2% up to 11,15%), for the scenarios with 300 message/second rate the *Fanout* exhibits less average power consumption (from 5% up to 31,18%). Initially this can look peculiar, however, by looking at the Table I, we see that in the scenarios with 300 messages/second rate, the *Fanout* scenarios' average Disk Access and average Memory Usage is significantly lower than the *Direct* scenarios' ones. For the scenarios with 100 messages/second rate, the average Disk Access and average Memory Usage look similar between the *Direct* and *Fanout* scenarios. The difference can affect the power consumption, and thus, the different behavior.

In some scenarios which were configured with a high incoming and outgoing message rates, as seen on Table I, RabbitMQ was not able to process the messages on time, and this resulted in the amount of messages consumed being less than expected and the memory and disk i/o to increase. Due to this stress on the messaging system, there were intermittent breaches of communication between RabbitMQ and Locust (i.e. Producers), lowering the rate of the latter. This especially happened for values 167 and 333 of the Outgoing/Incoming factor and for the cases where the incoming message rate was high (300 messages/sec). We still consider them as valid cases, as there are real scenarios for which the aforementioned issues can happen.

B. Comparison of Direct versus Fanout Strategies

Looking at all the scenarios of both use cases and comparing the *Direct* with *Fanout* scenarios, we see that *Fanout* scenarios in most of the cases have a lower power consumption than the *Direct* ones. Looking at the results on a more macroscopic level where economy of scale plays a major role, we could possibly be looking at vast differences with significant energy impact, especially when examining a multi-node system running 24/7.

Furthermore, looking at some specific scenarios of this experiment, we see that a small part of *Fanout* scenarios have more power consumption in comparison to the *Direct* cases. This means, that just by looking at the configuration, without taking into account metrics such as Disk Access and Memory usage together with Power Consumption, can lead to non optimal results.

To that end, we can conclude that in our Testbed for the *Account Driven* use case, it is more energy efficient to use a *Fanout* strategy instead of a *Direct* strategy. For the *Data Distribution Driven* use case, we see that the Input Message rate influences the power consumption significantly for the *Direct* strategy, while if faced with a higher message rate, the *Fanout* strategy is more efficient in terms of power consumption, again.

V. Conclusion

In this paper, we experimented with an existing RabbitMQ application, and we described and setup a Testbed in order to create project scenarios and benchmark our experiment strategies. The according dataset is made available to the community for further analysis.

The comparison was focused between 2 strategies: the *Fanout* and the *Direct* strategy applied to two common messaging use cases (*Account Driven* use case and *Data Distribution* use case) and examined whether they have significance difference in Power Consumption. We concluded that the comparison between the strategies mentioned did not have significant difference on a small scale, however at large scale and long term usage, where economy of scale is important, this difference can have a significant energy impact and thus lead to energy savings or waste. For the future, we anticipate to perform a deeper analysis of the data, in order to create a pricing model for messaging systems that takes into account

TABLE I: Message and Resource Usage Metrics in each of the Scenarios
(**Grey:** *Data Distribution* Driven Use Case, **White:** *Account Driven* Use Case)

Scenario Name	Expected Input Messages	Actual Input Messages	Expected Output Messages	Actual Output Messages	Outgoing/ Incoming factor	Avg Power Consumption (mW)	Avg Disk access (MB)	Avg Mem Usage (MB)
100_1_Direct_3_100	180.000	175.810	6.000.000	5.860.333	33	15691,76	0	248,39
100_1_Direct_3_500	180.000	177.767	30.000.000	29.627.833	167	124619,42	135,19	4704,67
100_1_Direct_3_1000	180.000	108.608	60.000.000	36.202.667	333	131837,93	126,79	7806,6
300_1_Direct_3_100	540.000	528.621	18.000.000	17.620.700	33	41587,06	0	305,1
300_1_Direct_3_500	540.000	241.746	90.000.000	40.291.000	167	177771,49	134,77	7208,88
300_1_Direct_3_1000	540.000	118.093	180.000.000	39.364.333	333	151291,26	158,84	10885,45
100_1_Fanout_3_100	180.000	175.650	6.000.000	5.855.000	33	15098,75	0	249,97
100_1_Fanout_3_500	180.000	175.683	30.000.000	29.280.500	167	127188,42	123,92	4706,97
100_1_Fanout_3_1000	180.000	119.549	60.000.000	39.849.667	333	148390,58	151,21	9384,35
300_1_Fanout_3_100	540.000	510.377	18.000.000	17.012.567	33	39603,3	0,01	297,52
300_1_Fanout_3_500	540.000	218.325	90.000.000	36.387.500	167	165369,74	130,29	6730,03
300_1_Fanout_3_1000	540.000	105.463	180.000.000	35.154.333	333	115327,5	83,69	7331,74
100_1_Direct_100_100	180.000	177.153	180.000	177.153	1	1513,34	0	185,87
100_1_Direct_500_500	180.000	177.757	180.000	177.757	1	2255,48	0	252,69
100_1_Direct_1000_1000	180.000	177.691	180.000	177.691	1	2829,67	0	302,39
300_1_Direct_100_100	540.000	528.899	540.000	528.899	1	3733,45	0	191,46
300_1_Direct_500_500	540.000	528.665	540.000	528.665	1	4622,04	0	274,09
300_1_Direct_1000_1000	540.000	528.830	540.000	528.830	1	5531,35	0	369,41
100_1_Fanout_100_100	180.000	175.580	180.000	175.580	1	1321,9	0	193,63
100_1_Fanout_500_500	180.000	155.013	180.000	155.013	1	2169,91	0	256,57
100_1_Fanout_1000_1000	180.000	162.409	180.000	162.409	1	2657,33	0	319,98
300_1_Fanout_100_100	540.000	509.578	540.000	509.578	1	3103,79	0	192,26
300_1_Fanout_500_500	540.000	528.573	540.000	528.573	1	4441,17	0,02	289,56
300_1_Fanout_1000_1000	540.000	515.405	540.000	515.405	1	5289,06	0	396,46

energy consumption. Furthermore, relevant performance models can be created that guide towards optimized configurations for a needed setup.

As a more general remark, in order to better enhance sustainability in large scale services like Cloud Computing, a similar approach could be applied to other types of services or architectures, leading to a user-level or action-level accountability on the energy usage of operations. This, if applied at a policy level, could motivate system engineers and architects to investigate such issues a priori and lead to a more sustainable software and system development.

REFERENCES

[1] AMQP 0-9-1 Specification — RabbitMQ — rabbitmq.com. https://www.rabbitmq.com/tutorials/amqp-concepts#exchanges. [Accessed 17-12-2024].

[2] GitHub - beltex/dshb: macOS system monitor — github.com. https://github.com/beltex/dshb. [Accessed 11-09-2024].

[3] GitHub - ColinIanKing/powerstat: Powerstat - power consumption measurement using stats Intel RAPL interface — github.com. https://github.com/ColinIanKing/powerstat. [Accessed 11-09-2024].

[4] GitHub - pika/pika: Pure Python RabbitMQ/AMQP 0-9-1 client library — github.com. https://github.com/pika/pika. [Accessed 13-12-2024].

[5] GitHub - rdegges/energy-tracker — github.com. https://github.com/rdegges/energy-tracker. [Accessed 11-09-2024].

[6] Grafana: The open observability platform — Grafana Labs — grafana.com. https://grafana.com/. [Accessed 11-09-2024].

[7] Green System Design - Setup Guide for ubuntu os. https://github.com/MariaVoreakou/Green-System-Design. [Accessed 06-12-2024].

[8] Intel Hyper-Threading — intel.com. https://www.intel.com/content/www/us/en/gaming/resources/hyper-threading.html. [Accessed 22-01-2025].

[9] Intel® Turbo Boost Technology — intel.com. https://www.intel.com/content/www/us/en/gaming/resources/turbo-boost.html. [Accessed 22-01-2025].

[10] Kepler as RPM — sustainable-computing.io. https://sustainable-computing.io/installation/kepler-rpm/. [Accessed 11-09-2024].

[11] Locust.io — locust.io. https://locust.io/. [Accessed 11-09-2024].

[12] NVIDIA Management Library (NVML) — developer.nvidia.com. https://developer.nvidia.com/management-library-nvml. [Accessed 22-01-2025].

[13] Open Hardware Monitor — openhardwaremonitor.org. https://openhardwaremonitor.org/. [Accessed 11-09-2024].

[14] OpenMessaging — openmessaging.cloud. https://openmessaging.cloud/. [Accessed 17-01-2025].

[15] Profiles :: Spring Boot — docs.spring.io. https://docs.spring.io/spring-boot/reference/features/profiles.html. [Accessed 11-09-2024].

[16] RabbitMQ Documentation — RabbitMQ — rabbitmq.com. https://www.rabbitmq.com/docs/use-rabbitmq. [Accessed 11-09-2024].

[17] Scaphandre Documentation — hubblo-org.github.io. https://hubblo-org.github.io/scaphandre-documentation/index.html. [Accessed 11-09-2024].

[18] A. Bagios. GitLab – Generalized Transmission Hub. https://gitlab.com/hua-dev/geth. [Accessed 11-09-2024].

[19] A. Bagios. Unified iot querying system. Master's thesis, Department Informatics and Telematics, Harokopio University of Athens, 2024. Available at https://estia.hua.gr/file/lib/default/data/29056/theFile.

[20] R. Buyya, S. Ilager, and P. Arroba. Energy-efficiency and sustainability in new generation cloud computing: A vision and directions for integrated management of data centre resources and workloads. *Software: Practice and Experience*, 54(1):24–38, 2024.

[21] E. Cadorel and D. Saingre. A protocol to assess the accuracy of process-level power models. In *2024 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 74–84. IEEE, 2024.

[22] G. Fieni, D. R. Acero, P. Rust, and R. Rouvoy. Powerapi: A python framework for building software-defined power meters. *Journal of Open Source Software*, 9(98):6670, 2024.

[23] V. Gudepu, R. R. Tella, C. Centofanti, J. Santos, A. Marotta, and K. Kondepu. Demonstrating the energy consumption of radio access networks in container clouds. In *NOMS2024, the IEEE/IFIP Network Operations and Management Symposium*, 2024.

[24] K. N. Haque, J. Islam, I. Ahmad, and E. Harjula. Decentralized pub/sub architecture for real-time remote patient monitoring: A feasibility study. In *Nordic Conference on Digital Health and Wireless Solutions*, pages 48–65. Springer, 2024.

[25] M. Jay, V. Ostapenco, L. Lefèvre, D. Trystram, A.-C. Orgerie, and B. Fichel. An experimental comparison of software-based power meters: focus on cpu and gpu. In *2023 IEEE/ACM 23rd International Symposium*

on Cluster, Cloud and Internet Computing (CCGrid), pages 106–118. IEEE, 2023.

[26] K. N. Khan, M. Hirki, T. Niemi, J. K. Nurminen, and Z. Ou. Rapl in action: Experiences in using rapl for power measurements. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems (TOMPECS)*, 3(2):1–26, 2018.

[27] C. Lin and M. Shahrad. Bridging the sustainability gap in serverless through observability and carbon-aware pricing. In *3rd Workshop on Sustainable Computer Systems (HotCarbon'24)*, page 41, 2024.

[28] Prometheus. Monitoring system & time series database — prometheus.io. https://prometheus.io/. [Accessed 09-01-2025].

[29] L.-D. Radu. Green cloud computing: A literature survey. *Symmetry*, 9(12), 2017.

[30] V. D. Reddy, B. Setz, G. S. V. Rao, G. Gangadharan, and M. Aiello. Metrics for sustainable data centers. *IEEE Transactions on Sustainable Computing*, 2(3):290–303, 2017.

[31] RobBagby. Queue-Based Load Leveling pattern - Azure Architecture Center — learn.microsoft.com. https://learn.microsoft.com/en-us/azure/architecture/patterns/queue-based-load-leveling. [Accessed 13-12-2024].

[32] E. Sahafizadeh and B. Tork Ladani. A model for social communication network in mobile instant messaging systems. *IEEE Transactions on Computational Social Systems*, 7(1):68–83, 2020.

[33] T. Strempel. Measuring the energy consumption of software written in c on x86-64 processors. 2021.

[34] thingsboard. ThingsBoard — Open-source IoTPlatform — thingsboard.io. https://thingsboard.io/. [Accessed 14-12-2024].